

Título

Análise e Validação do Modelo PINN

Este notebook tem como objetivo carregar o modelo PINN de Black-Scholes previamente treinado e avaliá-lo em um novo conjunto de dados (PETR4 2024) que não foi visto durante o treinamento (out-of-sample).

Objetivos:

1. **Validar a Acurácia:** Verificar o desempenho de precificação do modelo em dados novos.
2. **Analisar a Volatilidade:** Extrair a superfície de volatilidade implícita e o *smile* para o novo ativo.
3. **Verificar as Gregas:** Analisar o comportamento do Delta ($\partial V / \partial S$) aprendido.
4. **Gerar Insights:** Produzir gráficos e métricas de alta qualidade para inclusão em um artigo acadêmico.

1. Configuration for Evaluation

a. Import and Paths

```
# Imports and paths
import os
import json
import time
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 3D plotting
from mpl_toolkits.mplot3d import Axes3D

# tqdm (opcional para loops longos)
from tqdm import tqdm

# Scipy for implied vol solver
from scipy.optimize import brentq

# Torch and project imports
import torch
from torch.utils.data import TensorDataset, DataLoader

# Para importa a classe/modelo e funções de processamento
from src.model import PINN_BlackScholes
```

```

from src.config import MODEL_CONFIG, PATHS, DATA_CONFIG,
TRAINING_CONFIG
from src.data_loader import carregar_e_processar_dados_opcoes,
carregar_taxa_juros

# Carregar Paths
ROOT = os.getcwd() # ou caminho do projeto
PATH_RESULTS = os.path.join(ROOT, 'resultados')
PATH_MODEL = os.path.join(PATH_RESULTS, 'modelo_final')
TRAIN_HISTORY_PATH = os.path.join(PATH_RESULTS,
'training_history.csv')
MODEL_WEIGHTS_PATH = os.path.join(PATH_MODEL,
'best_model_weights.pth')
DATA_STATS_PATH = os.path.join(PATH_MODEL, 'data_stats.json')

# Carregar Dados brutos
RAW_DATA_DIR = os.path.join(ROOT, 'dados', 'brutos')
RAW_2024_DIR = os.path.join(RAW_DATA_DIR, 'Databricks')

# Carregar data_stats
with open(DATA_STATS_PATH, 'r') as f:
    data_stats = json.load(f)
data_stats

# Instancia modelo
# Ajusta config para consistência (mantém problem_type do config
original)
model_config = MODEL_CONFIG.copy()
# Garantir input_size correto (já definido no config)
device = torch.device(TRAINING_CONFIG['device'] if 'device' in
TRAINING_CONFIG else 'cpu')
model = PINN_BlackScholes(config=model_config, data_stats=data_stats)
model.to(device)

# Carrega pesos
state = torch.load(MODEL_WEIGHTS_PATH, map_location=device)
model.load_state_dict(state)
model.eval()
print("Ok!")

Ok!

```

b. Helpers: Black-Scholes price & implied vol solver

```

# Helpers BS price and implied vol (European call)
# Small, robust implementations for Black-Scholes European call price
and implied vol

from math import log, sqrt, exp

```

```

from scipy.stats import norm

def bs_call_price(S, K, T, r, sigma):
    """
    Black-Scholes European call price (scalar or numpy arrays).
    Assumes T is time-to-maturity in years (T>0). If T==0 returns
    intrinsic value.
    """
    S = np.asarray(S, dtype=float)
    K = np.asarray(K, dtype=float)
    T = np.asarray(T, dtype=float)
    r = np.asarray(r, dtype=float)
    sigma = np.asarray(sigma, dtype=float)
    price = np.zeros_like(S, dtype=float)
    # handle T==0 (payoff)
    mask_T0 = (T <= 0)
    price[mask_T0] = np.maximum(S[mask_T0] - K[mask_T0], 0.0)
    mask = ~mask_T0
    if mask.any():
        St = S[mask]; Kt = K[mask]; Tt = T[mask]; rt = r[mask]; sig = sigma[mask]
        d1 = (np.log(St/Kt) + (rt + 0.5*sig*sig)*Tt) /
        (sig*np.sqrt(Tt))
        d2 = d1 - sig*np.sqrt(Tt)
        price[mask] = St*norm.cdf(d1) - Kt*np.exp(-rt*Tt)*norm.cdf(d2)
    return price

def implied_volatility_call(market_price, S, K, T, r,
sigma_bounds=(1e-6, 3.0)):
    """
    Solve implied volatility via brentq on difference BS_price -
    market_price.
    Returns NaN if solver fails.
    """
    market_price = float(market_price)
    if market_price <= max(0.0, S - K*np.exp(-r*T)):
        # price below intrinsic lower bound -> implied vol ~ 0
        return 0.0
    def price_diff(sig):
        return float(bs_call_price(np.array([S]), np.array([K]),
np.array([T]), np.array([r]), np.array([sig]))[0]) - market_price
    a, b = sigma_bounds
    try:
        root = brentq(price_diff, a, b, maxiter=200, xtol=1e-6)
        return float(root)
    except Exception:
        return float('nan')

print("Ok!")

```

c. Processar e Preparar Dados de 2021-2023 (Treinamento) e 2024 (Validação)

```
# Carrega e processa dados usando seu data_loader (mantendo preprocess original)
# Função wrapper que produz X (normalized) e y (premium), e DataFrame original com colunas úteis
def prepare_dataset_from_folder(folder_path, df_juros=None, max_samples=None):
    """
    Procura arquivos CSV na pasta, concatena, processa e retorna X, y, df_proc.
    Usa carregar_e_processar_dados_opcoes do projeto (espera dados no formato do repositório).
    """
    # Reutiliza função do projeto que provavelmente exige um arquivo consolidado.
    # Se sua função precisa de caminho específico, ajuste conforme necessário.
    # Aqui assumimos que carregar_e_processar_dados_opcoes aceita (raw_data_path, df_juros)
    df = carregar_e_processar_dados_opcoes(folder_path, df_juros)
    if df is None or df.empty:
        raise RuntimeError(f"Nenhum dado processado em {folder_path}")
    if max_samples is not None and len(df) > max_samples:
        df = df.sample(n=max_samples, random_state=42)
    # Normalização usando data_stats (mesmas transformações do treino)
    S_min, S_max = data_stats['S_min'], data_stats['S_max']
    K_min, K_max = data_stats['K_min'], data_stats['K_max']
    T_max = data_stats['T_max']
    df['S_norm'] = (df['spot_price'] - S_min) / (S_max - S_min)
    df['K_norm'] = (df['strike'] - K_min) / (K_max - K_min)
    df['T_norm'] = df['time_to_maturity'] / T_max
    # Colunas de entrada esperadas:
    ['S_norm', 'K_norm', 'T_norm', 'r', 'premium']
    X =
df[['S_norm', 'K_norm', 'T_norm', 'r', 'premium']].values.astype(float)
y = df[['premium']].values.astype(float)
return X, y, df

# Carregar taxas de juros (se necessário) – sua função já implementa carregar_taxa_juros
df_juros = carregar_taxa_juros(PATHS['selic_data'])
# Preparar dataset de treino (2021-2023)
X_train, y_train, df_train = prepare_dataset_from_folder(RAW_DATA_DIR, df_juros)
print("Train:", len(df_train), "amostras")
# Preparar dataset 2024 (validacao)
X_val, y_val, df_val = prepare_dataset_from_folder(RAW_2024_DIR,
```

```
df_juros)
print("Val 2024:", len(df_val), "amostras")

Carregando dados da taxa de juros de: d:\UERJ\Programacao_e_Codigos\
PINN_Opcoes_BR\dados\brutos\taxa_selic.csv
Dados da taxa de juros carregados com sucesso.
Encontrados 26 arquivos de ativos para processar...

Processando arquivos de ativos: 100%|██████████| 26/26 [00:10<00:00,
2.55it/s]

Unificando todos os dataframes de ativos...
Realizando engenharia de features...
Processamento concluído. DataFrame final com 998004 amostras.
Train: 998004 amostras
Encontrados 20 arquivos de ativos para processar...

Processando arquivos de ativos: 100%|██████████| 20/20 [00:06<00:00,
2.99it/s]

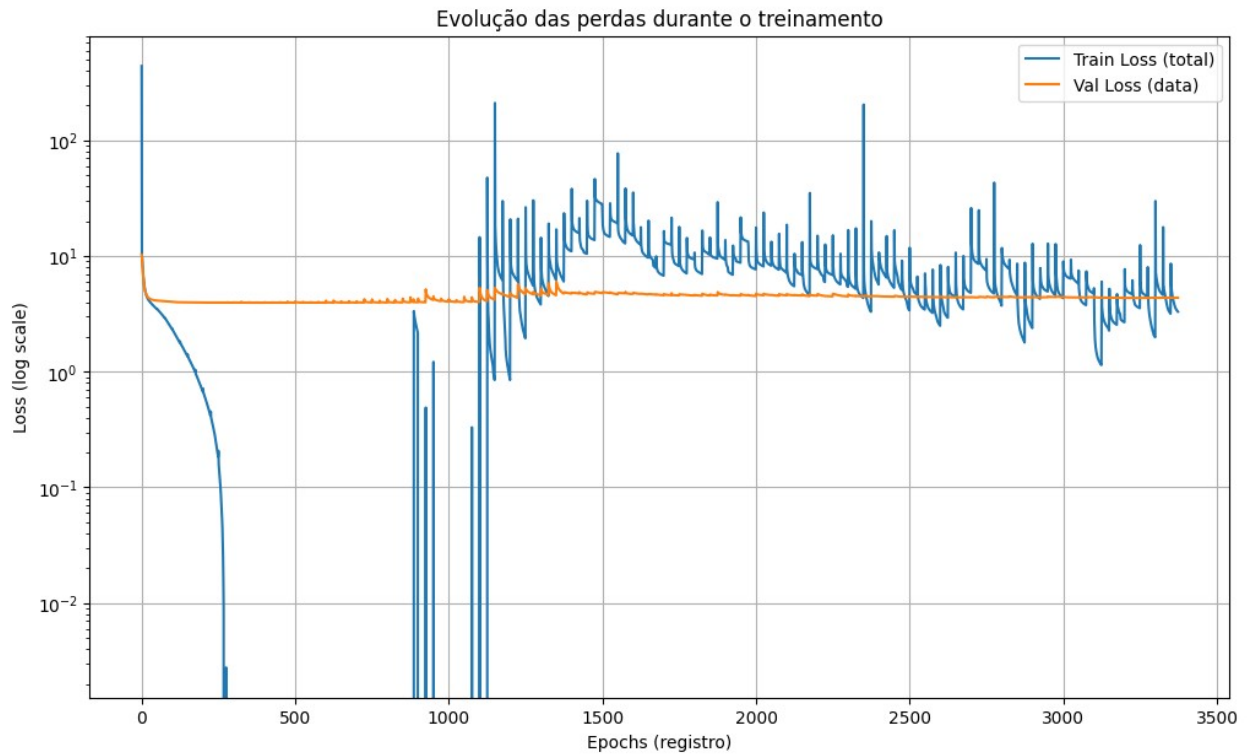
Unificando todos os dataframes de ativos...
Realizando engenharia de features...
Processamento concluído. DataFrame final com 702881 amostras.
Val 2024: 702881 amostras
```

2. Evaluation of Training History

Evolução das perdas durante o treinamento

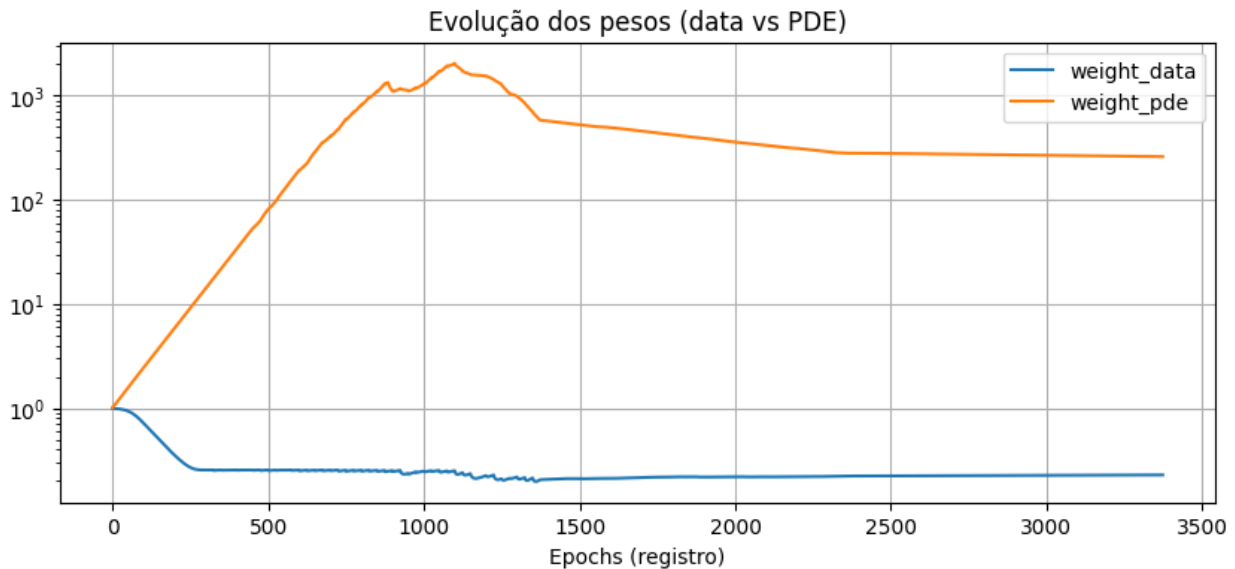
```
# Cell 3 - Carrega histórico de treino e plota curvas simples
history = pd.read_csv(TRAIN_HISTORY_PATH)
history.head()

# Plots das losses
plt.figure(figsize=(12, 7))
plt.plot(history['train_loss'], label='Train Loss (total)')
plt.plot(history['val_loss'], label='Val Loss (data)')
plt.yscale('log')
plt.xlabel('Epochs (registro)')
plt.ylabel('Loss (log scale)')
plt.legend()
plt.title('Evolução das perdas durante o treinamento')
plt.grid(True)
plt.show()
```



Evolução dos pesos (data vs PDE) (Escala Ajustada com log)

```
# Plot dos pesos
if 'weight_data' in history.columns and 'weight_pde' in
history.columns:
    plt.figure(figsize=(10,4))
    plt.plot(history['weight_data'], label='weight_data')
    plt.plot(history['weight_pde'], label='weight_pde')
    plt.yscale('log')
    plt.xlabel('Epochs (registro)')
    plt.legend()
    plt.title('Evolução dos pesos (data vs PDE)')
    plt.grid(True)
    plt.show()
```



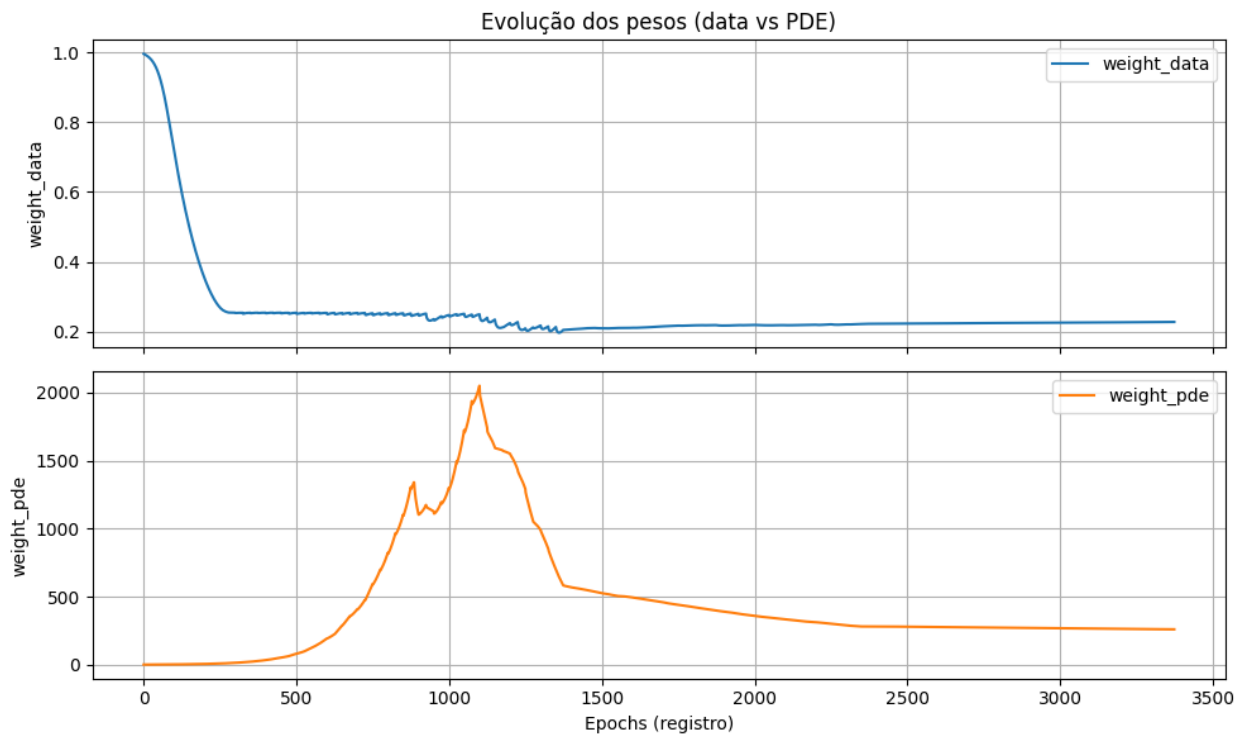
Evolução dos pesos (data vs PDE) (Subplot dos pesos)

```
# Plot dos pesos com subplots
if 'weight_data' in history.columns and 'weight_pde' in
history.columns:
    # 1. Criar a figura com 2 linhas, 1 coluna, compartilhando o eixo
    X
    fig, (ax1, ax2) = plt.subplots(
        nrows=2,
        ncols=1,
        figsize=(10, 6), # Um pouco mais alto para caber os dois plots
        sharex=True      # Eixo X compartilhado
    )

    # 2. Plotar 'weight_data' no gráfico de cima (ax1)
    ax1.plot(history['weight_data'], label='weight_data',
color='tab:blue')
    ax1.set_ylabel('weight_data')
    ax1.legend()
    ax1.grid(True)
    ax1.set_title('Evolução dos pesos (data vs PDE)') # Título no plot
superior

    # 3. Plotar 'weight_pde' no gráfico de baixo (ax2)
    ax2.plot(history['weight_pde'], label='weight_pde',
color='tab:orange')
    ax2.set_ylabel('weight_pde')
    ax2.set_xlabel('Epochs (registro)') # Rótulo do X só no plot
inferior
    ax2.legend()
    ax2.grid(True)
```

```
# 4. Mostrar o gráfico
plt.tight_layout() # Ajusta o espaçamento
plt.show()
```



Inferência (preço e sigma)

```
# Inferência em lote (usa DataLoader para evitar estourar memória)
def infer(model, X_np, batch_size=4096, device=device):
    model.eval()
    dataset = TensorDataset(torch.from_numpy(X_np).float())
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=False,
num_workers=0)
    prices = []
    sigmas = []
    with torch.no_grad():
        for (xb,) in loader:
            xb = xb.to(device)
            out = model(xb)
            prices.append(out['price'].cpu().numpy())
            sigmas.append(out['sigma'].cpu().numpy())
    prices = np.vstack(prices).reshape(-1)
    sigmas = np.vstack(sigmas).reshape(-1)
    return prices, sigmas

# Inferência train (in-sample)
price_train_pred, sigma_train_pred = infer(model, X_train)
price_val_pred, sigma_val_pred = infer(model, X_val)
```


Inferência (preço e sigma) quero Comparar as previsões do treinamento e da validação com os dados

Métricas (train in-sample)

```
# Métricas de preço
def metrics(y_true, y_pred):
    y_true = np.asarray(y_true).reshape(-1)
    y_pred = np.asarray(y_pred).reshape(-1)
    mse = np.mean((y_true - y_pred)**2)
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_true - y_pred))
    return {'mse': mse, 'rmse': rmse, 'mae': mae}

metrics_train = metrics(y_train, price_train_pred)
metrics_val = metrics(y_val, price_val_pred)

print(f"{'Metric':<10} | {'Train':<10} | {'Validation ':<10}")
print("-" * 35)
for k in metrics_train.keys():
    print(f"{k.upper():<10} | {metrics_train[k]:<10.5f} | {metrics_val[k]:<10.5f}")
```

Metric	Train	Validation
MSE	3.97397	2.82364
RMSE	1.99348	1.68037
MAE	0.89466	0.82197

Calcular volatilidade implícita (a partir do prêmio de mercado) e comparar com sigma_pred

```
# Calcula implied vol para cada amostra (pode ser lento; use tqdm)
def compute_implied_vols(df_input, price_col='premium'):
    n = len(df_input)
    ivs = np.zeros(n)
    for i in tqdm(range(n), desc='Calc IVs'):
        S = float(df_input['spot_price'].iloc[i])
        K = float(df_input['strike'].iloc[i])
        T = float(df_input['time_to_maturity'].iloc[i])
        r = float(df_input['r'].iloc[i])
        mprice = float(df_input[price_col].iloc[i])
        ivs[i] = implied_volatility_call(mprice, S, K, T, r)
    return ivs

# Compute for validation set only (2024) – can also do for train if desired
print("Calculating implied vols for validation set (this may take a
```

```

while)...")
iv_val = compute_implied_vols(df_val)          # array shape (n_val,)
# For train (optional): iv_train = compute_implied_vols(df_train)

# Compare predicted sigma (model output) vs implied vol
# Note: model sigma may be local vol scale; ensure units consistent
# (both in decimal annual)
from sklearn.metrics import mean_squared_error
mask_valid_iv = ~np.isnan(iv_val)
mse_vol = mean_squared_error(iv_val[mask_valid_iv],
sigma_val_pred[mask_valid_iv])
print("MSE between sigma_pred and implied_vol (val):", mse_vol)

Calculating implied vols for validation set (this may take a while)...
Calc IVs: 100%|██████████| 702881/702881 [10:32<00:00, 1111.98it/s]
MSE between sigma_pred and implied_vol (val): 5754.671057436389

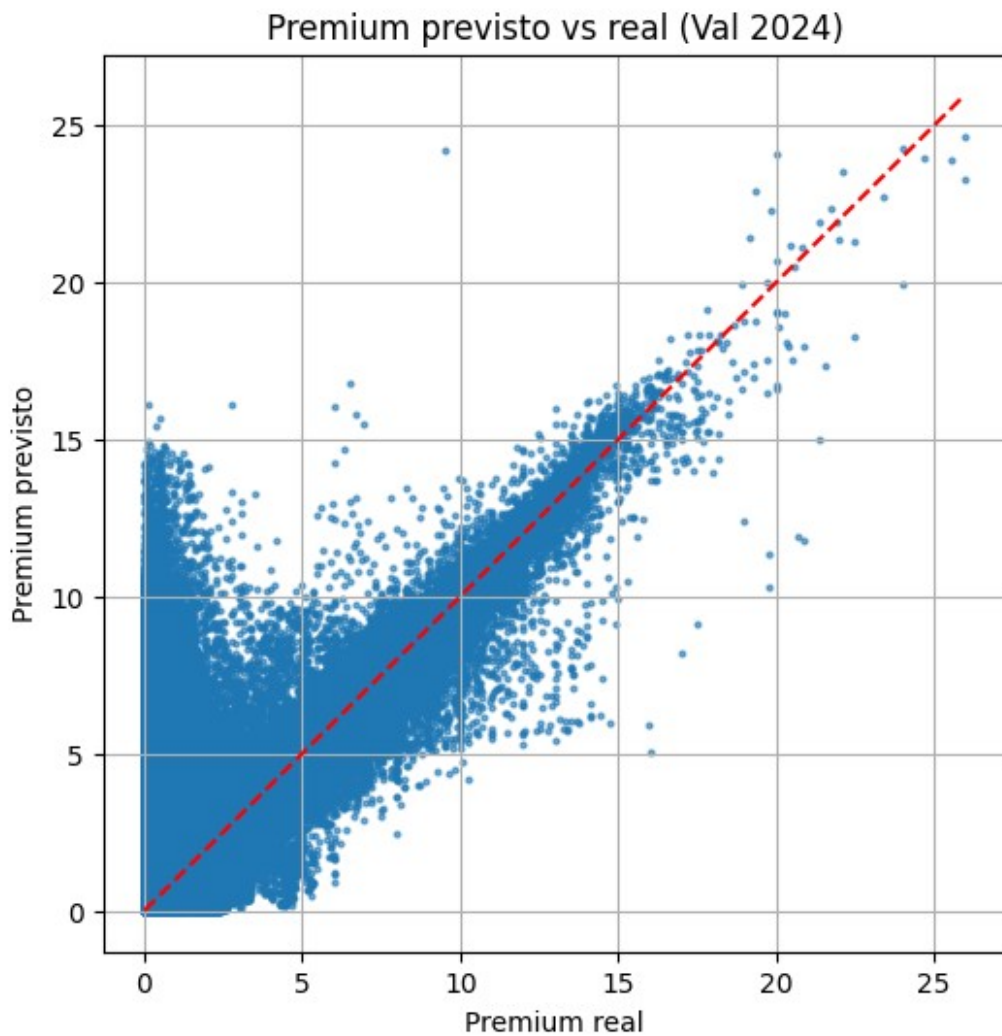
```

Plots: price predicted vs real (scatter)

```

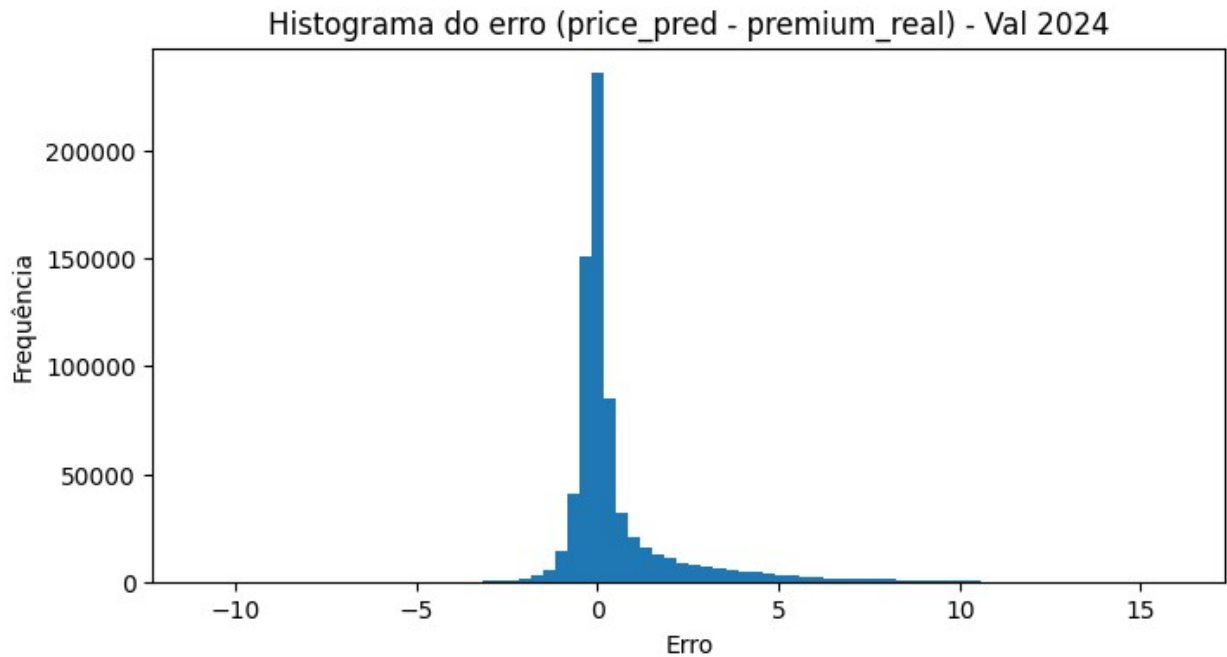
# Scatter price_pred vs real (validation)
plt.figure(figsize=(6,6))
plt.scatter(y_val.reshape(-1), price_val_pred, s=4, alpha=0.6)
plt.plot([y_val.min(), y_val.max()], [y_val.min(), y_val.max()],
'r--')
plt.xlabel('Premium real')
plt.ylabel('Premium previsto')
plt.title('Premium previsto vs real (Val 2024)')
plt.grid(True)
plt.show()

```



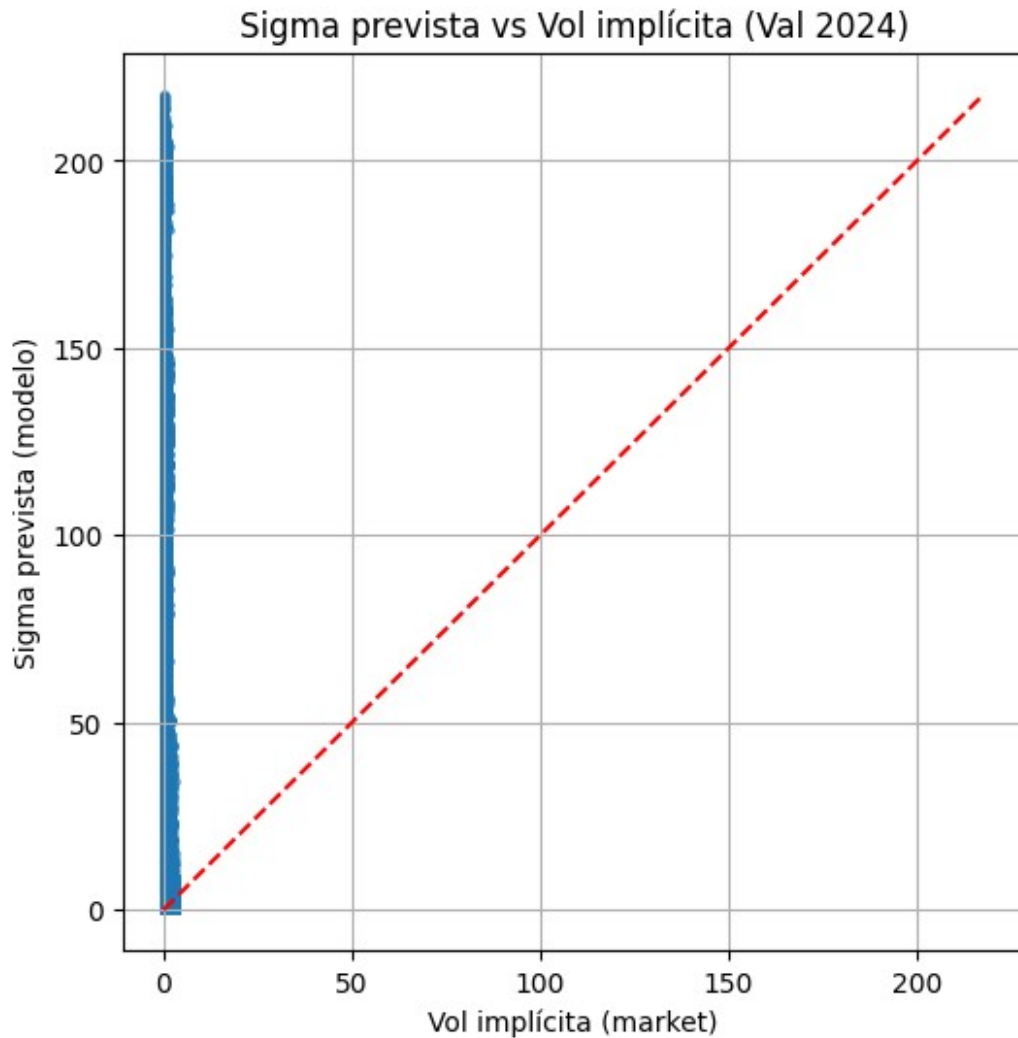
Plots: residual histogram

```
# Residual histogram
resid = (price_val_pred - y_val.reshape(-1))
plt.figure(figsize=(8,4))
plt.hist(resid, bins=80)
plt.title('Histograma do erro (price_pred - premium_real) - Val 2024')
plt.xlabel('Erro')
plt.ylabel('Frequência')
plt.show()
```



Plots: sigma_pred vs implied_vol (scatter)

```
# Volatility comparison
plt.figure(figsize=(6,6))
plt.scatter(iv_val[mask_valid_iv], sigma_val_pred[mask_valid_iv], s=6,
alpha=0.6)
plt.plot([0, max(np.nanmax(iv_val), np.nanmax(sigma_val_pred))], [0,
max(np.nanmax(iv_val), np.nanmax(sigma_val_pred))], 'r--')
plt.xlabel('Vol implícita (market)')
plt.ylabel('Sigma prevista (modelo)')
plt.title('Sigma prevista vs Vol implícita (Val 2024)')
plt.grid(True)
plt.show()
```



Plots: estatísticas

```
# Estatísticas de erro na volatilidade
diff_vol = sigma_val_pred[mask_valid_iv] - iv_val[mask_valid_iv]
print("Vol diff mean:", np.nanmean(diff_vol), "std:",
      np.nanstd(diff_vol))
print("Vol MSE:", np.nanmean(diff_vol**2))
```

```
Vol diff mean: 50.74390618230028 std: 56.38906846897093
Vol MSE: 5754.671057436389
```

Heatmap de erro por moneyness x tempo (Val)

```
# Heatmap de erro por moneyness x T
df_val['price_pred'] = price_val_pred
df_val['error_abs'] = np.abs(df_val['price_pred'] - df_val['premium'])
# moneyness
df_val['moneyness'] = df_val['spot_price'] / df_val['strike']
```

```

# bins
df_val['moneyness_bin'] = pd.cut(df_val['moneyness'],
bins=np.linspace(0.5,1.5,21))
df_val['T_bin'] = pd.cut(df_val['time_to_maturity'], bins=10)

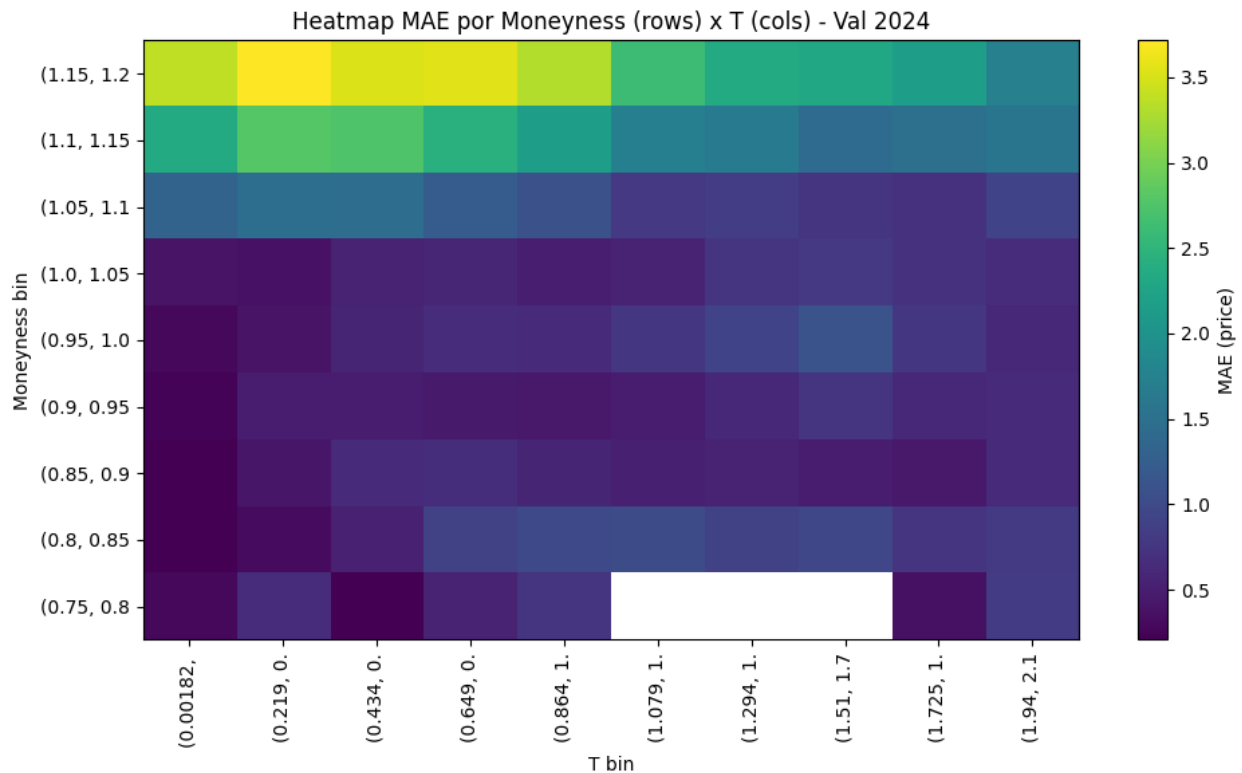
pivot = df_val.pivot_table(index='moneyness_bin', columns='T_bin',
values='error_abs', aggfunc='mean')
plt.figure(figsize=(10,6))
plt.imshow(pivot.values, origin='lower', aspect='auto',
cmap='viridis')
plt.colorbar(label='MAE (price)')
plt.title('Heatmap MAE por Moneyness (rows) x T (cols) - Val 2024')
plt.xlabel('T bin')
plt.ylabel('Moneyness bin')
plt.xticks(ticks=np.arange(pivot.shape[1]), labels=[str(c)[:10] for c
in pivot.columns], rotation=90)
plt.yticks(ticks=np.arange(pivot.shape[0]), labels=[str(i)[:10] for i
in pivot.index])
plt.tight_layout()
plt.show()

```

```

C:\Users\Rubens Molina\AppData\Local\Temp\
ipykernel_41500\2982546239.py:10: FutureWarning: The default value of
observed=False is deprecated and will change to observed=True in a
future version of pandas. Specify observed=False to silence this
warning and retain the current behavior
    pivot = df_val.pivot_table(index='moneyness_bin', columns='T_bin',
values='error_abs', aggfunc='mean')

```

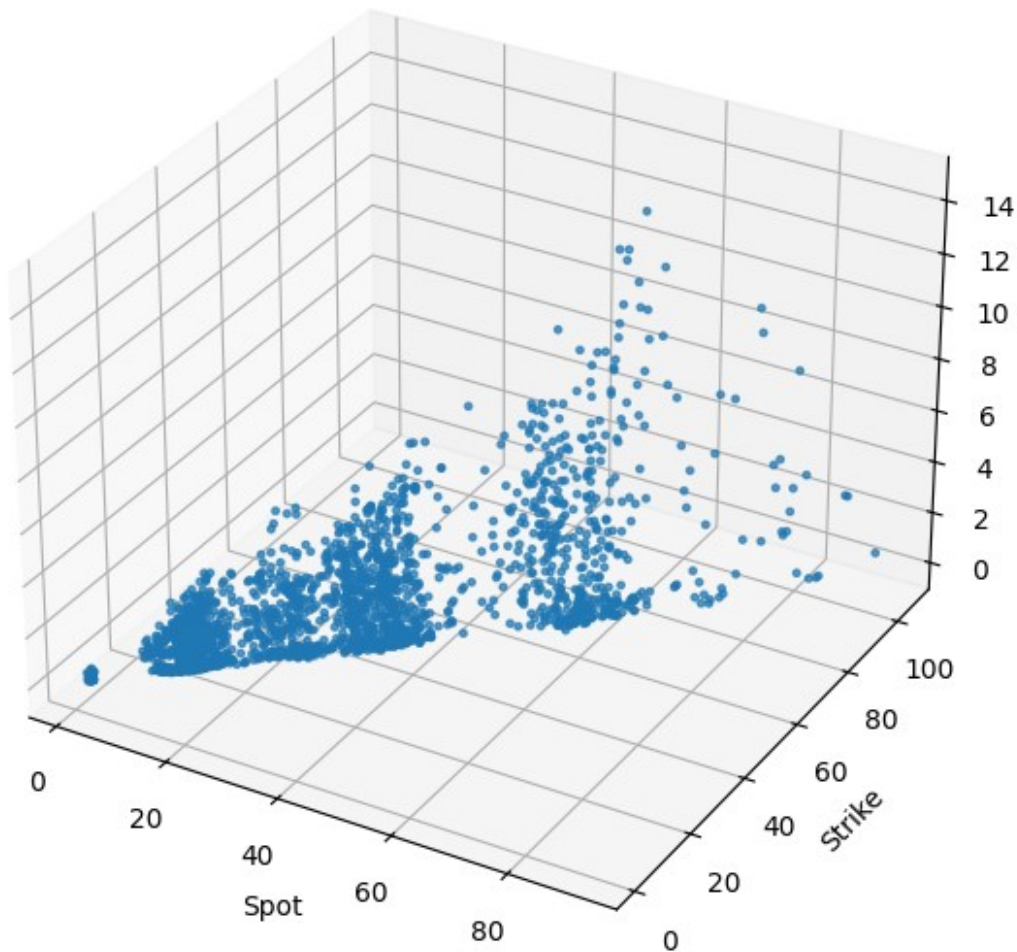


Superfície de preço prevista (amostra)

```
# Superfície price(S,K) para um tempo fixed (amostra)
# Escolha um T index médio e construa superfície
sample_df = df_val.copy()
# filtra por maturidade próxima de median
T_med = sample_df['time_to_maturity'].median()
sample_sub = sample_df[np.abs(sample_df['time_to_maturity'] - T_med) <
(0.02 * T_med)]
# Take subset for plotting
plot_df = sample_sub.sample(n=min(2000, len(sample_sub)),
random_state=42)

fig = plt.figure(figsize=(10,7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(plot_df['spot_price'], plot_df['strike'],
plot_df['price_pred'], s=6, alpha=0.7)
ax.set_xlabel('Spot')
ax.set_ylabel('Strike')
ax.set_zlabel('Price_pred')
ax.set_title(f'Superfície Price_pred (samples) - T approx
{T_med:.3f}')
plt.show()
```

Superfície Price_pred (samples) - T approx 0.111



Aggregated metrics & export de resultados para o artigo

```
# Métricas agregadas e export
metrics_summary = {
    'train_price_mse': metrics_train['mse'],
    'train_price_rmse': metrics_train['rmse'],
    'train_price_mae': metrics_train['mae'],
    'val_price_mse': metrics_val['mse'],
    'val_price_rmse': metrics_val['rmse'],
    'val_price_mae': metrics_val['mae'],
    'vol_val_mse': float(mse_vol)
}
metrics_summary

{'train_price_mse': np.float64(3.9739703424197934),
 'train_price_rmse': np.float64(1.9934819644079536),
```



```
'train_price_mae': np.float64(0.8946563627799117),  
'val_price_mse': np.float64(2.82364262354684),  
'val_price_rmse': np.float64(1.6803697877392463),  
'val_price_mae': np.float64(0.8219652795526315),  
'vol_val_mse': 5754.671057436389}
```

Salvando os resultados de validação

```
# Salva CSV com previsões e erros (útil para tabelas e análises no  
artigo)  
out_val = df_val.copy()  
out_val['price_pred'] = price_val_pred  
out_val['sigma_pred'] = sigma_val_pred  
out_val['implied_vol'] = iv_val  
out_val['error_price'] = out_val['price_pred'] - out_val['premium']  
out_val.to_csv(os.path.join(PATH_RESULTS, 'val_predictions.csv'),  
index=False)  
print("Arquivo salvo:", os.path.join(PATH_RESULTS,  
'val_predictions.csv'))
```

Arquivo salvo: d:\UERJ\Programacao_e_Codigos\PINN_Opcoes_BR\
resultados\val_predictions.csv